

JACOBIAN AND VELOCITIES FOR 6-DOF MANIPULATORS WITH OFFSETS AND SPHERICAL WRISTS

ABSTRACT

This research presents a comprehensive analysis of the differential kinematics for a specific 6-Degree of Freedom (DOF) Kuka serial manipulator. By defining the robot's geometry through standard Denavit-Hartenberg (DH) parameters, we derive the forward kinematics model essential for state estimation. The core of this study focuses on the numerical computation of the Jacobian matrix, employing a finite difference approach to approximate the linear mapping between joint space velocities and task space end-effector velocities. Special attention is given to the derivation of angular velocity components using the properties of skew-symmetric matrices and the time derivative of rotation matrices. Furthermore, the practical utility of the Jacobian is demonstrated through a differential inverse kinematics control scheme, enabling precise Cartesian trajectory tracking. Detailed MATLAB implementations are dissected to illustrate the algorithmic translation of these theoretical concepts into functional simulation code.

1. INTRODUCTION

This research document analyzes the differential kinematics of a specific 6-Degree of Freedom (DOF) serial manipulator (Kuka-type). The focus is on the Jacobian Matrix, a critical mathematical tool that relates joint space velocities to task space (end-effector) velocities.

We will analyze the physical parameters of the robot, derive the mathematical foundation of the Numerical Jacobian, and provide a detailed, line-by-line illustration of the MATLAB implementation used for both Jacobian calculation and Inverse Kinematics control.

2. ROBOT DEFINITION AND KINEMATICS

2.1 Denavit-Hartenberg (DH) Parameters

The robot is defined using the Standard Denavit-Hartenberg parameters. These parameters describe the transformation from one link to the next.

$link(i)$	θ_i	d_i	a_i	α_i
1	θ_1	$d1$	$a1$	90°
2	$\theta_2 + 90^\circ$	0	$a2$	-180°
3	θ_3	0	$a3$	-90°
4	θ_4	$d2$	0	90°
5	θ_5	0	0	-90°
6	θ_6	$d3$	0	0°

2.2 Forward Kinematics (FK)

The position and orientation of the end-effector are determined by multiplying the transformation matrices of individual links:

$$T_{global} = A_1(q_1) \cdot A_2(q_2) \cdot A_3(q_3) \cdot A_4(q_4) \cdot A_5(q_5) \cdot A_6(q_6)$$

Where each A_i is a 4×4 homogeneous transformation matrix derived from row i of Table 1. The result is:

$$T = \begin{bmatrix} R_{3 \times 3} & P_{3 \times 1} \\ 0_{1 \times 3} & 1 \end{bmatrix}$$

3. THEORETICAL FOUNDATION: THE JACOBIAN

The Jacobian matrix serves as the fundamental mathematical bridge between the robot's internal joint space and the external Cartesian task space. It acts as a linear mapping operator that transforms joint velocities ($\dot{q}$) directly into the end-effector's linear and angular velocities (v, ω) by representing the matrix of partial derivatives, the Jacobian effectively linearizes the complex, non-linear forward kinematics equations around a specific operating point. This linearization is critical for differential control strategies, allowing us to calculate how infinitesimal changes in joint angles propagate to the tool tip's motion. Furthermore, the Jacobian provides essential insights into the mechanical structure, helping to identify singular configurations where the robot loses degrees of freedom.

3.1 What is the Jacobian Matrix?

To understand the Jacobian matrix, we must first revisit the fundamental concept of the derivative in a univariate context. Consider a single-variable function $f(x) = x^2$. The derivative, denoted as $\frac{df}{dx}$, represents the instantaneous rate of change of the function with respect to its input. For this specific function, the derivative is:

$$\frac{df}{dx} = 2x$$

This scalar value quantifies how the output responds to infinitesimal perturbations in the input x .

- **Multivariable Scalar Functions:**

Multivariable scalar functions expand into the multivariable domain; we encounter functions that accept multiple inputs simultaneously. Consider a scalar function f dependent on variables x_1 and x_2 defined as:

$$f(x_1, x_2) = x_1^2 + 3x_1x_2 + x_2^2$$

In this context, we utilize the partial derivative to isolate the sensitivity of the function to a single variable while holding others constant. The gradient of this function is composed of its partial derivatives with respect to x_1 and x_2 .

$$\frac{\partial f}{\partial x_1} = 2x_1 + 3x_2 \quad , \quad \frac{\partial f}{\partial x_2} = 3x_1 + 2x_2$$

- **Vector-Valued Functions and the Jacobian:**

The Jacobian matrix arises when we generalize this concept to vector-valued functions systems involving multiple input variables and multiple output functions. Let $f(x)$ be a function mapping a vector $x \in R^n$ to a vector $f \in R^m$. Consider the specific system defined by the vector function

$$f(x) = \begin{bmatrix} f_1(x_1, x_2) \\ f_2(x_1, x_2) \end{bmatrix} = \begin{bmatrix} x_1^2 + x_2 \\ x_1 + x_2^2 \end{bmatrix}$$

The Jacobian matrix, denoted as J , represents the linear mapping of the system's first-order partial derivatives. It describes how the output vector changes relative to changes in the input vector. For a system with two inputs and two outputs, the Jacobian is defined as:

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{bmatrix}$$

By computing the partial derivatives for the system defined above:

$$\text{For } f_1: \frac{\partial f_1}{\partial x_1} = 2x_1 \text{ and } \frac{\partial f_1}{\partial x_2} = 1$$

$$\text{For } f_2: \frac{\partial f_2}{\partial x_1} = 1 \text{ and } \frac{\partial f_2}{\partial x_2} = 2x_2$$

Substituting these terms into the matrix yields the final Jacobian for the system:

$$J(x) = \begin{bmatrix} 2x_1 & 1 \\ 1 & 2x_2 \end{bmatrix}$$

3.2 Jacobian In robotics

In the domain of robotics, the Jacobian matrix (J) is not merely a mathematical abstraction but the fundamental operator connecting Joint Space (the internal configuration of motors and actuators) to the Task Space (the external position and orientation of the end-effector).

While Forward Kinematics defines a static relationship between joint angles and end-effector position, the Jacobian defines the dynamic relationship, mapping velocities and forces.

In the multivariable calculus section, we defined the Jacobian as a matrix of partial derivatives. In robotics, we apply this exact definition to motion.

The Core Equation:

$$v = J(q) \times \dot{q}$$

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ w_x \\ w_y \\ w_z \end{bmatrix} = \begin{bmatrix} \frac{\partial x}{\partial q_1} & \cdots & \frac{\partial x}{\partial q_6} \\ \frac{\partial y}{\partial q_1} & \cdots & \frac{\partial y}{\partial q_6} \\ \frac{\partial z}{\partial q_1} & \cdots & \frac{\partial z}{\partial q_6} \\ \frac{\partial w_x}{\partial q_1} & \cdots & \frac{\partial w_x}{\partial q_6} \\ \frac{\partial w_y}{\partial q_1} & \cdots & \frac{\partial w_y}{\partial q_6} \\ \frac{\partial w_z}{\partial q_1} & \cdots & \frac{\partial w_z}{\partial q_6} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \\ \dot{q}_4 \\ \dot{q}_5 \\ \dot{q}_6 \end{bmatrix}$$

Where:

- v is the End-Effector Velocity Vector (6×1). It contains 3 linear velocities (v_x, v_y, v_z) and 3 angular velocities ($\omega_x, \omega_y, \omega_z$).
- $J(q)$ is the Jacobian Matrix (6×6 for your robot). It changes based on the robot's current pose q .
- $\dot{q}$ is the Joint Velocity Vector (6×1). It represents how fast each motor is spinning ($\dot{q}_1, \dot{q}_2, \dots, \dot{q}_6$).

What is Physical Interpretation for this part?

Each column i of the Jacobian tells you: "If I move only joint i at 1 rad/s, what is the resulting velocity vector of the end-effector?"

3.2.1 Calculating the Jacobian

While the Jacobian can be derived analytically, a robust method for computational implementation is the Numerical differentiation (Finite Difference) approach. This method approximates the partial derivatives by introducing a minute perturbation (δ) to each joint and observing the resulting change in the end-effector's pose.

- **Step 1: Compute Initial State**

First, we compute the Forward Kinematics (FK) for the current joint configuration q . This yields the homogeneous transformation matrix T_{curr}

$$T_{curr} = FK(q) = \begin{bmatrix} R_{curr} & P_{curr} \\ 0 & 1 \end{bmatrix}$$

Where P_{curr} is the position vector and R_{curr} is the rotation matrix.

- **Step 2: Apply Perturbation**

To find the partial derivative with respect to joint i , we add a small perturbation δ (e.g., 10^{-6}) to that specific joint while keeping others constant:

$$q_{pert} = q + [\delta, 0, 0, 0, 0, 0]$$

- **Step 3: Compute Perturbed State**

We calculate the new Forward Kinematics based on the perturbed joints

$$T_{pert} = FK(q_{pert}) = \begin{bmatrix} R_{pert} & T_{pert} \\ 0 & 1 \end{bmatrix}$$

- **Step 4: Calculate Linear Velocity Component (J_v)**

The linear velocity component of the Jacobian column is the rate of change of the position vector:

$$J_{vi} \approx \frac{P_{pert} - P_{curr}}{\delta}$$

This approximates the derivative $\frac{\partial P}{\partial q_i}$

- **Step 5: Calculate Angular Velocity Component (J_ω)**

Determining angular velocity requires analyzing the change in the Rotation Matrix. We compute the derivative of the rotation matrix $\dot{R}$

$$\dot{R} \approx \frac{R_{pert} - R_{curr}}{\delta}$$

Using the property that $\dot{R} = S(\omega)R$, where $S(\omega)$ is the skew-symmetric matrix of the angular velocity vector, we can isolate $S(\omega)$

$$S(\omega) = \dot{R}R^T$$

The resulting 3×3 Matrix S takes the form

$$S(\omega) = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$$

By applying the inverse operation (often denoted as the vex operator), we extract the angular velocity vector

$$J\omega_i = [\omega_x, \omega_y, \omega_z]^T$$

Step 6: Construct the Column

The final i -th column of the Jacobian is the concatenation of the linear and angular components:

$$j_i = \begin{bmatrix} j_{vi} \\ j_{wi} \end{bmatrix}$$

This process is repeated for all n joints to construct the full $6 \times n$ Jacobian matrix.

3.2.2 Inverse Kinematics Control (Resolved Rate Motion Control)

The Jacobian is instrumental in trajectory planning. While Forward Kinematics maps joint angles to positions, Inverse Kinematics typically seeks to find the joint angles required to reach a specific target. By utilizing the Jacobian, we can control the robot at the velocity level.

From the relationship $v_e = J(q)\dot{q}$, we can solve for the required joint velocities $\dot{q}$ to achieve a desired end-effector velocity $v_{desired}$:

$$\dot{q} = j^{-1}(q) \times v_{desired}$$

In practice, particularly for 6-DOF robots where singularities may occur, the Moore-Penrose Pseudoinverse ($J^\dagger$) is preferred over the standard inverse to ensure numerical stability:

$$\dot{q} = J^\dagger(q) \times v_{desired}$$

3.2.3 Algorithm Implementation

The implementation is developed in MATLAB. The simulation utilizes a numerical differentiation method to compute the Jacobian matrix dynamically at every time step, ensuring adaptability to the 6-DOF KUKA manipulator structure defined in the previous sections.

Block 1: Kinematic Modeling (DH Parameters)

The robot is modeled using the Denavit-Hartenberg (DH) convention. The link parameters (twist α , length a , offset d , and angle θ) are defined to match the KUKA manipulator specifications.

```
a1 = 37.5; a2 = 160; a3 = 15;
d1 = 135.8; d2 = 138.1; d3 = 28.2;

L(1) = Link([0, d1, a1, pi/2]);
L(2) = Link([pi/2, 0, a2, -pi]);
L(3) = Link([0, 0, a3, -pi/2]);
L(4) = Link([0, d2, 0, pi/2]);
L(5) = Link([0, 0, 0, -pi/2]);
L(6) = Link([0, d3, 0, 0]);

Kuka = SerialLink(L, 'name', 'My_Kuka');
```

Block 2: Initialization and Target Definition

The simulation initializes the robot at a non-singular starting posture $q_{current}$. The control target is defined not as a final position, but as a velocity vector $V_{desired} \in R^6$, comprising linear velocity (v_x, v_y, v_z) and angular velocity ($\omega_x, \omega_y, \omega_z$).

```
% Initial Joint Configuration (Home Position)
q_current = deg2rad([0, 90, 0, 0, 0, 0]);

% Desired Twist Vector [vx; vy; vz; wx; wy; wz]
V_desired = [30; -30; -50; 1; 0.5; 0];

% Time Integration Settings
dt = 0.05;
total_time = 10.0;
steps = total_time / dt;
delta_math = 1e-6;
```

Block 3: The Control Loop & Numerical Jacobian

The core of the algorithm runs inside a temporal loop. At each step t , the Jacobian matrix $J(q)$ is computed using the Finite Difference Method. Each joint is perturbed by a small delta (δ), and the resulting change in the end-effector pose is measured to approximate the partial derivatives.

Once J is found, the required joint velocities are calculated using the Moore-Penrose pseudoinverse:

$$\dot{q} = J^{\dagger}(q) \times v_{desired}$$

At the end

$$q_{current} = q_{current} + (\dot{q}_{dot}' * dt);$$

```
for t = 0:dt:total_time

    J_numerical = zeros(6, 6);
    T0 = double(Kuka.fkine(q_current));
    R0 = T0(1:3,1:3);
    P0 = T0(1:3,4);

    for i = 1:6
        q_pert = q_current;
        q_pert(i) = q_pert(i) + delta_math;

        T1 = double(Kuka.fkine(q_pert));
        R1 = T1(1:3,1:3);
        P1 = T1(1:3,4);

        J_numerical(1:3, i) = (P1 - P0) / delta_math;
        dR = (R1 - R0) / delta_math;
        J_numerical(4:6, i) = Vex(dR * R0');

    end

    q_dot = pinv(J_numerical) * V_desired;

    q_current = q_current + (q_dot' * dt);
```

“You are taking the instantaneous velocity ($\dot{q}$), pretending it stays constant for a tiny fraction of a second (dt), and "teleporting" the robot to that new spot.”